

CONTRACT NOTE REWORK

Data Model

DynamoDB Table & Record Reference

Estimates 1-4

Overview

This document describes the DynamoDB data model behind the contract note rework. It is a companion to the estimate walkthroughs and the API specification, giving the concrete record shapes, key patterns, and indexes the implementation will use.

The design follows a single-table pattern per bounded area. The bulk of the system, templates, sections, shared sections, rules, versions, data source attachments, and bespoke contracts, lives in one ContractNoteTemplates table, distinguished by partition key (PK) and sort key (SK) patterns. DocuSign envelope tracking sits in its own DocuSignEnvelopes table.

Larger blobs are kept out of DynamoDB: pdf-me section layouts (schema JSON) and rendered PDFs live in S3, referenced from the records by their S3 key.

Reading the key patterns

Values in braces are substituted at runtime, e.g. TEMPLATE#{templateId} becomes TEMPLATE#4f3c... . Records that share a partition key (a template and its sections, versions, rule, and change log) are stored together and retrieved with a single query, which is the core idea behind single-table design.

Tables at a glance

TABLE	ESTIMATES	HOLDS
ContractNoteTemplates	1, 3b, 4	Templates, sections, shared sections, rules, version history, change log, data source attachments, and all bespoke records
DocuSignEnvelopes	2	One record per DocuSign envelope, for status tracking and traceability

Estimate 1 - Templates & Sections

The core template records. Everything owned by a template shares the TEMPLATE#{templateId} partition, so a single query returns the template with its sections, rule, and change log.

Table: `ContractNoteTemplates`

Template		
PK	TEMPLATE#{templateId}	SK METADATA
ATTRIBUTE	TYPE	DESCRIPTION
templateId	String	UUID
name	String	Template display name (unique)
description	String	Template description
priority	Number	Evaluation priority (1 = highest)
sectionCount	Number	Denormalised count of sections
createdAt	String	ISO 8601 timestamp
updatedAt	String	ISO 8601 timestamp
createdBy	String	Cognito username

Section (template-owned)

PK TEMPLATE#{templateId}

SK SECTION#{sortOrder}#{sectionId}

The sort order is baked into the SK so sections return already ordered.

ATTRIBUTE	TYPE	DESCRIPTION
sectionId	String	UUID
name	String	Section display name
sortOrder	Number	Position within the template
isShared	Boolean	Whether this references a shared section
sharedSectionId	String	(optional) Reference to a shared section
schemaS3Key	String	S3 key for the schema JSON
createdAt	String	ISO 8601 timestamp
updatedAt	String	ISO 8601 timestamp

Rule

PK TEMPLATE#{templateId}

SK RULE

The specification tree that selects this template (see the Estimate 1 walkthrough).

ATTRIBUTE	TYPE	DESCRIPTION
specification	Map	JSON specification tree
updatedAt	String	ISO 8601 timestamp
updatedBy	String	Cognito username

Template Change Log

PK TEMPLATE#{templateId}

SK CHANGELOG#{timestamp}

ATTRIBUTE	TYPE	DESCRIPTION
changeType	String	section-added, section-removed, section-reordered, metadata-updated, rule-updated
description	String	Human-readable description of the change
createdAt	String	ISO 8601 timestamp
createdBy	String	Cognito username

Global Secondary Indexes

INDEX	GSI PK	GSI SK	ENABLES
PriorityIndex	ALL_TEMPLATES (constant)	priority (Number)	Query all templates ordered by priority in a single call

Estimate 1 - Shared Sections & Versions

Reusable sections live under their own partition and are linked to templates by reference records. Section edits are versioned rather than overwritten.

Table: `ContractNoteTemplates`

Shared Section		
PK	SHARED_SECTION#{sectionId}	SK METADATA
ATTRIBUTE	TYPE	DESCRIPTION
sectionId	String	UUID
name	String	Section display name
isTermsAndConditions	Boolean	Whether this is a T&C section
schemaS3Key	String	S3 key for the schema JSON
createdAt	String	ISO 8601 timestamp
updatedAt	String	ISO 8601 timestamp
createdBy	String	Cognito username

Shared Section Reference		
PK	SHARED_SECTION#{sectionId}	SK REF#{templateId}
One per template using the shared section; how references are tracked and counted.		
ATTRIBUTE	TYPE	DESCRIPTION
templateId	String	Template using this shared section
templateName	String	Denormalised for display

Section Version

PK

SECTION_VERSION#{sectionId}

SK

VERSION#{timestamp}

Every schema save appends a version; nothing is overwritten, enabling history and revert.

ATTRIBUTE	TYPE	DESCRIPTION
versionId	String	UUID
sectionId	String	Section this version belongs to
schemaS3Key	String	S3 key for this version's schema JSON
createdAt	String	ISO 8601 timestamp
createdBy	String	Cognito username
description	String	(optional) Change description

Estimate 3b - Data Source Attachments

Data source extensibility adds two record types to the same table. No new table is needed. Attachments hang off the template; dependencies hang off the shared section.

Table: `ContractNoteTemplates`

Template Data Source

PK

TEMPLATE#{templateId}

SK

DATASOURCE#{database}#{tableName}

Marks a Glue table as attached to a template; its columns become available as fields.

ATTRIBUTE	TYPE	DESCRIPTION
database	String	Glue database name
tableName	String	Glue table name
displayName	String	User-friendly name (defaults to table name)
attachedAt	String	ISO 8601 timestamp
attachedBy	String	Cognito username

Shared Section Dependency

PK

SHARED_SECTION#{sectionId}

SK

DATASOURCE_DEP#{database}#{tableName}

Auto-tracked from the data source fields a shared section uses; drives the missing-dependency prompt.

ATTRIBUTE	TYPE	DESCRIPTION
database	String	Glue database name
tableName	String	Glue table name

Estimate 4 - Bespoke Contracts

Bespoke records mirror the template pattern but under a BESPOKE#{bespokeId} partition, keeping one-off documents fully isolated from standard templates. A pending record is written first by the pipeline skip.

Table: `ContractNoteTemplates`

Pending Bespoke Request		
PK	BESPOKE_PENDING#{brytNumber}	SK OFFER#{offerReference}
Written by the render pipeline when it skips a bespoke-flagged customer.		
ATTRIBUTE	TYPE	DESCRIPTION
brytNumber	String	Customer BrytNumber
offerReference	String	Offer reference from the contract data
customerName	String	Customer name
contractDataS3Key	String	S3 key of the original contract JSON
receivedAt	String	ISO 8601 timestamp when received
status	String	pending

Bespoke Contract Note

PK	BESPOKE#{bespokeId}	SK	METADATA
----	---------------------	----	----------

ATTRIBUTE	TYPE	DESCRIPTION
bespokeId	String	UUID
brytNumber	String	Customer BrytNumber
offerReference	String	Offer reference
customerName	String	Customer name
contractDataS3Key	String	S3 key of the contract JSON
clonedFromTemplateId	String	(optional) Template ID if cloned
status	String	draft, rendering, rendered, failed
currentRenderS3Key	String	(optional) S3 key of latest rendered PDF
currentRenderVersion	Number	Current render version number
docusignEnvelopeId	String	(optional) Envelope ID if sent
docusignStatus	String	(optional) sent, completed, declined, expired
createdAt / updatedAt	String	ISO 8601 timestamps
createdBy / updatedBy	String	Cognito usernames

Bespoke Section

PK BESPOKE#{bespokeId}

SK SECTION#{sortOrder}#{sectionId}

Same shape as a template section; independent copies when cloned from a template.

ATTRIBUTE	TYPE	DESCRIPTION
sectionId	String	UUID
name	String	Section display name
sortOrder	Number	Position within the bespoke contract
isShared	Boolean	Whether this references a shared section
sharedSectionId	String	(optional) Reference to a shared section
schemaS3Key	String	S3 key for the schema JSON
versionNumber	Number	Current version number
createdAt / updatedAt	String	ISO 8601 timestamps

Bespoke Render History

PK BESPOKE#{bespokeId}

SK RENDER#{version}

Append-only; one per on-demand render.

ATTRIBUTE	TYPE	DESCRIPTION
version	Number	Render version number
pdfS3Key	String	S3 key of the rendered PDF
renderedAt	String	ISO 8601 timestamp
renderedBy	String	Cognito username
status	String	success, failed
errorMessage	String	(optional) Error details if failed

Estimate 2 - DocuSign Envelopes

DocuSign tracking uses a separate table. One record per envelope, looked up by envelope ID during webhook processing and by Salesforce reference for debugging.

Table: `DocuSignEnvelopes`

Envelope		
PK	ENVELOPE#{envelopeId}	SK METADATA
ATTRIBUTE	TYPE	DESCRIPTION
<code>envelopeId</code>	<i>String</i>	DocuSign envelope ID
<code>salesforceRef</code>	<i>String</i>	Customer Salesforce reference
<code>contractNoteS3Key</code>	<i>String</i>	S3 key of the original contract note PDF
<code>customerEmail</code>	<i>String</i>	Email the envelope was sent to
<code>customerName</code>	<i>String</i>	Customer name on the envelope
<code>status</code>	<i>String</i>	sent, delivered, completed, declined, expired
<code>signedPdfS3Key</code>	<i>String</i>	(optional) S3 key of signed PDF once downloaded
<code>createdAt</code>	<i>String</i>	ISO 8601 timestamp of creation
<code>updatedAt</code>	<i>String</i>	ISO 8601 timestamp of last status update
<code>errorMessage</code>	<i>String</i>	(optional) Error / decline reason

Global Secondary Indexes

INDEX	GSI PK	GSI SK	ENABLES
SalesforceRefIndex	<code>salesforceRef</code>	<code>createdAt</code>	Query all envelopes for a given customer (for debugging)

What lives in S3 (not DynamoDB)

Two kinds of larger data are deliberately kept in S3 and referenced from DynamoDB by key: the pdf-me section layouts (schema JSON, one object per section and per version),

and the rendered and signed PDFs. This keeps DynamoDB items small and cheap to query while letting the bulkier content sit in cost-effective object storage.

- › Schema JSON - referenced by schemaS3Key on sections, shared sections, and section versions
- › Rendered contract note PDFs - the render pipeline output (Estimate 1)
- › Signed PDFs - stored after DocuSign completion (Estimate 2), keyed by Salesforce ref + envelope ID
- › Bespoke render PDFs - referenced by pdfS3Key on render history records (Estimate 4)
- › Original contract JSON - stored for bespoke customers, referenced by contractDataS3Key

A note on scope

These record shapes come from the estimate design documents and are intended to convey intent and structure. Exact attribute names, optionality, and any additional GSIs may be refined during implementation without changing the overall approach.